EA-0046 - AQELYN Engineering Archive

EA-0046 - AQELYN Control Validation & Continuous Assurance Engine

Implementation Specification: IS-046

Publication Standard: FULL_COMPLETE

Repository Structure: Immutable

Status: Published Engineering Archive

Section 001 - Document Control

Field	Value
Project	AQELYN
Engineering Archive	EA-0046
Implementation Specification	IS-046
Title	AQELYN Control Validation & Continuous Assurance Engine
Date	2026-07-08
Source of Truth	Master Markdown

Section 002 - Revision History

Revision History for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 003 - Executive Summary

This archive defines the AQELYN Control Validation & Continuous Assurance Engine. The engine validates whether security controls operate as intended using tests, evidence, metrics, and continuous assurance workflows. It is specified as a first-class capability within the AQELYN Cyber Security Operating Environment and is designed to integrate with the AQELYN Kernel, Universal Object Model, Event Bus, Evidence Engine, Knowledge Graph, Trust Engine, Mission Engine, Workflow Engine, Policy Engine, and prior engineering archives.

Section 004 - Vision

The vision for AQELYN Control Validation & Continuous Assurance Engine is to provide a deterministic, evidence-backed, event-driven capability that can be implemented by engineering teams without redesigning the fixed AQELYN repository or previously approved architecture.

Section 005 - Purpose

The purpose of this engine is to validates whether security controls operate as intended using tests, evidence, metrics, and continuous assurance workflows. It provides implementation-ready guidance for components, interfaces, lifecycle behavior, security controls, validation, traceability, and publication artifacts.

Section 006 - Scope

In scope: platform services, canonical objects, events, API contracts, security controls, operational telemetry, validation, and evidence. Out of scope: replacing external vendor platforms, redesigning repository structure, or bypassing approved AQELYN subsystems.

Section 007 - Engineering Objectives

- OBJ-0046-001: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 1.
- OBJ-0046-002: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 2.
- OBJ-0046-003: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 3.
- OBJ-0046-004: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 4.
- OBJ-0046-005: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 5.
- OBJ-0046-006: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 6.
- OBJ-0046-007: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 7.
- OBJ-0046-008: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 8.
- OBJ-0046-009: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 9.
- OBJ-0046-010: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 10.
- OBJ-0046-011: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 11.
- OBJ-0046-012: Provide a verifiable capability boundary for aqelyn control validation & continuous assurance engine objective 12.

Section 008 - Definitions & Terminology

Key terms are defined using AQELYN terminology: canonical object, immutable event, evidence reference, policy result, trust context, mission context, remediation workflow, operational finding, and engineering archive.

Section 009 - System Context

System Context for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 010 - Architectural Overview

Architectural Overview for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 011 - Internal Architecture

Internal Architecture for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 012 - Component Specifications

Discovery Manager

Discovery Manager is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Normalization Service

Normalization Service is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Repository Service

Repository Service is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Assessment Engine

Assessment Engine is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Policy Adapter

Policy Adapter is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Trust Adapter

Trust Adapter is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Evidence Linker

Evidence Linker is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Recommendation Engine

Recommendation Engine is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Event Publisher

Event Publisher is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Analytics Service

Analytics Service is responsible for the relevant subset of AQELYN Control Validation & Continuous Assurance Engine behavior. It exposes versioned interfaces, produces structured telemetry, and maintains traceability to requirements.

Section 013 - Universal Object Model Extensions

Universal Object Model Extensions for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 014 - Lifecycle Model

Lifecycle Model for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 015 - Governance Model

Governance Model for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 016 - Trust Model

Trust Model for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 017 - Policy Model

Policy Model for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 018 - Event Model

- ControlDiscovered
- ControlUpdated
- ControlAssessmentCompleted
- ControlRiskDetected
- ControlPolicyViolationDetected
- ControlRecommendationGenerated
- ControlWorkflowRequested

• ControlEvidenceLinked

• ControlArchived

Every event uses the AQELYN Universal Event Model with event id, type, version, timestamp, correlation id, source engine, severity, evidence references, trust context, mission context, policy references, and digital signature.

Section 019 - Internal Interface Contracts

Internal Interface Contracts for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 020 - External Interface Contracts

External Interface Contracts for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 021 - Security Architecture

Security Architecture for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 022 - Threat Model

Threat Model for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 023 - Cryptographic Architecture

Cryptographic Architecture for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 024 - Performance Architecture

Performance Architecture for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 025 - Scalability Architecture

Scalability Architecture for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 026 - Reliability & High Availability

Reliability & High Availability for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 027 - Observability Architecture

Observability Architecture for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 028 - Logging Architecture

Logging Architecture for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 029 - Monitoring & Operational Metrics

Monitoring & Operational Metrics for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 030 - Engineering Constraints

Engineering Constraints for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 031 - Testing Strategy

Testing Strategy for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 032 - Validation & Acceptance Criteria

Validation & Acceptance Criteria for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 033 - Requirements Matrix

Requirement ID	Capability	Priority	Verification
REQ-FR-0046-001	Discovery and Intake	Mandatory	Integration Test
REQ-FR-0046-002	Normalization	Mandatory	Functional Test
REQ-FR-0046-003	Inventory	Mandatory	System Test
REQ-FR-0046-004	Assessment	Mandatory	Unit Test
REQ-FR-0046-005	Policy Evaluation	Mandatory	Integration Test
REQ-FR-0046-006	Trust Integration	Mandatory	Functional Test
REQ-FR-0046-007	Evidence Linkage	Mandatory	System Test
REQ-FR-0046-008	Recommendation	Mandatory	Unit Test
REQ-FR-0046-009	Event Publication	Mandatory	Integration Test
REQ-FR-0046-010	Historical Analytics	Mandatory	Functional Test
REQ-FR-0046-011	API Access	Mandatory	System Test
REQ-FR-0046-012	Workflow Integration	Mandatory	Unit Test

Section 034 - Traceability Matrix

Requirement	Architecture Component	Implementation Module	Verification	Evidence
REQ-FR-0046-001	ARC-0046-001 Discovery and Intake Component	MOD-0046-001	Integration Test	EV-0046-001 Evidence Package
REQ-FR-0046-002	ARC-0046-002 Normalization Component	MOD-0046-002	Functional Test	EV-0046-002 Evidence Package
REQ-FR-0046-003	ARC-0046-003 Inventory Component	MOD-0046-003	System Test	EV-0046-003 Evidence Package
REQ-FR-0046-004	ARC-0046-004 Assessment Component	MOD-0046-004	Unit Test	EV-0046-004 Evidence Package
REQ-FR-0046-005	ARC-0046-005 Policy Evaluation Component	MOD-0046-005	Integration Test	EV-0046-005 Evidence Package
REQ-FR-0046-006	ARC-0046-006 Trust Integration Component	MOD-0046-006	Functional Test	EV-0046-006 Evidence Package
REQ-FR-0046-007	ARC-0046-007 Evidence Linkage Component	MOD-0046-007	System Test	EV-0046-007 Evidence Package
REQ-FR-0046-008	ARC-0046-008 Recommendation Component	MOD-0046-008	Unit Test	EV-0046-008 Evidence Package
REQ-FR-0046-009	ARC-0046-009 Event Publication Component	MOD-0046-009	Integration Test	EV-0046-009 Evidence Package
REQ-FR-0046-010	ARC-0046-010 Historical Analytics Component	MOD-0046-010	Functional Test	EV-0046-010 Evidence Package
REQ-FR-0046-011	ARC-0046-011 API Access Component	MOD-0046-011	System Test	EV-0046-011 Evidence Package
REQ-FR-0046-012	ARC-0046-012 Workflow Integration Component	MOD-0046-012	Unit Test	EV-0046-012 Evidence Package

Section 035 - Engineering Journal

Engineering decisions for EA-0046: preserve immutable repository structure; use master Markdown as source of truth; expose versioned APIs; produce evidence-backed events; maintain deterministic rule evaluation; generate PDF, HTML, README, manifest, diagrams, examples, and final ZIP from the approved master.

Section 036 - Example Artifacts

Example artifacts include sample event payloads, requirement evidence packets, traceability extracts, configuration profiles, assessment reports, remediation records, and operational dashboard exports.

Section 037 - Publication Manifest

The EA-0046 package contains Master Markdown, PDF, HTML, README, manifest.json, requirements matrix, traceability matrix, engineering journal, example artifacts, five SVG diagrams, and final FULL_COMPLETE ZIP.

Section 038 - References

References for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.

Section 039 - Engineering Review

Engineering review confirms completeness, standards alignment, traceability coverage, event-driven integration, immutable repository compliance, and publication readiness.

Section 040 - Publication Readiness

Publication Readiness for AQELYN Control Validation & Continuous Assurance Engine defines implementation guidance required for coding, validation, operations, and maintenance.

The design is modular, deterministic, evidence-backed, and compatible with the fixed AQELYN repository structure.

All implementation units shall map to approved requirements, versioned interfaces, verification activities, and auditable evidence artifacts.

Security, observability, reliability, and policy behavior shall be implemented consistently across environments and deployment models.

Implementation teams shall preserve separation of concerns between ingestion, normalization, assessment, decisioning, event publication, and workflow orchestration.

Every persisted record shall include provenance, source metadata, version metadata, lifecycle state, and references to the evidence used to justify the current state.

Operational behavior shall be testable through deterministic fixtures, replayable event streams, synthetic provider inputs, and failure injection scenarios.

The component boundary shall remain compatible with future plug-ins and shall not require modification of the fixed AQELYN repository structure.